

Academic Year	Module	Portfolio Number	Assessment Type
2026	6CS030: Artificial Intelligence and Machine Learning	3	Report

Part - III - Language Tasks.

Student ID: 2408028

Members : Luniva Shrestha

Course: 6CS012

University email address: L.Shretha7@wlv.ac.uk

Tutor: Ms. Sunita Parajuli

Course Leader: Mr. Siman Giri

This project deals with the task of automatically distinguishing between genuine and fake news articles by applying deep learning methods. The problem of detecting fake news can be considered quite significant since fake news is widely spread through social media channels as well as online news portals, shaping public opinion in various ways and leading to social, political, and economic problems. In order to address the research problem three deep learning models were used, which included Simple RNN, Long Short-Term Memory Network (LSTM), and LSTM model incorporating pre-trained embeddings GloVe word embedding. Several data pre-processing steps were conducted on the dataset, such as text cleaning, conversion to lowercase, expanding contractions, removing stopwords, lemmatization, tokenizing, and padding. The results showed that LSTM with GloVe embeddings had the best accuracy score, equal to 98.55%. Although the Simple RNN model was quite computationally efficient, it failed to understand the context of the article. Standard LSTM solved the vanishing gradient problem and was able to learn longer sequences. The research demonstrates the importance of utilizing advanced architecture of neural networks and pre-trained embeddings for solving text classification problems.

Table of Contents

Table of Contents	3
1. Title.....	6
2. Introduction.....	7
3. Dataset.....	8
3.1. Source:	8
3.2 Data Size:.....	8
Dataset Split:	8
3.3. Preprocessing:.....	8
4. Methodology.....	11
4.1. Text Preprocessing:.....	11
4.2. Simple RNN model:.....	12
4.2.1. Model Architecture:.....	12
4.2.2. Loss Function:	12
4.2.3. Optimizer:.....	12
4.2.4. Hyperparameters:	12
4.3. LSTM Model.....	13
4.3.1. Architecture	13
4.3.2. Loss Function:	13
4.3.3. Optimizer.....	13
4.3.4. Hyperparameters.....	13
4.4. Transfer Learning with GloVe Embeddings	13
4.4.1. Pretrained Model	13
4.4.2. Approach	14
4.4.3. Loss Function	14
4.4.4. Optimizer.....	14
4.4.5. Hyperparameters.....	14
5. Experiments and Results	15
5.1. RNN vs LSTM Performance	15
5.2. Computational Efficiency	16
5.3. Training with Different Embeddings	17
5.4. Model Evaluation	18
5.5. Graphical User Interface (GUI).....	21
6. Conclusion and Future Work.....	22
6.1. Important Findings	22
6.2. Limitations	22
6.3. Future Work.....	22
7. References	23

Figure 1 Class Distribution of True and Fake News Samples (Count and Percentage)	9
Figure 2 Word CCloud Representation of Most Frequent Words in True and Fake News Articles	9
Figure 3 Top 20 Most Frequent Words in True and Fake News Articles After Preprocessing.....	9
Figure 4 Distribution of News Article Sequence Lengths Before Padding	10
Figure 5 Training and Validation Accuracy and Loss for Simple RNN Model	15
Figure 6 Training and Validation and Loss for LSTM Model.....	16
Figure 7 Training and Validation Accuracy and Loss for LSTM with GLove Embeddings	16
Figure 8 All 3 models validation accuracy on one plot	17
Figure 9 Prediction breakdown of the simple RNN showing correct and incorrect classification on the test	19
Figure 10classification results of the LSTM illustrating its performance across true and fake news categories	19
Figure 11 Confusion matrix for LSTM with GLove Embeddings	20
Figure 12 Test Accuracy chart.....	20
Figure 13 Gradio based Fake News Detection GUI showing article input and multi-model classification output	21

1. Title

Classification of Fake News using Deep Learning Techniques with RNN, LSTM, and Pretrained GloVe Embedding

2. Introduction

With the expansion of digital media and social networking platforms, the spread of false information has become more common, highlighting the importance of developing reliable fake news detection systems. Fake news tends to distort reality and lead to a lot of misconceptions, and it can have adverse impacts on social, political, and economic frameworks. The creation of automated mechanisms to detect and identify fake news has thus become critical. As such, fake news detection is an important field of study within data mining (Shu et al., 2017).

While machine learning models generally require human intervention in formulating features, they fail to take into consideration many semantic relationships in text data. Unlike traditional approaches, deep learning methods including RNN and LSTM architectures can automatically learn meaningful patterns from text, making them effective for classification problems.

RNN algorithms are adept at handling sequential information, but their major drawback lies in the vanishing gradient problem when capturing longer sequences. This problem can be solved by incorporating the gated network structure found in LSTM networks. In addition to these methods, pre-trained embeddings like GloVe give word vectors with semantic representations based on large data sets.

This study involves implementing and analyzing three deep learning models for fake news classification:

- Simple RNN model utilizing learnable embedding vectors
- LSTM architecture with trainable word embeddings
- LSTM model initialized with pre-trained GloVe embeddings

The objective of this work is to evaluate the effectiveness, computational efficiency, and classification performance of these models for detecting fake news articles.

3. Dataset

3.1. Source:

This study utilized the True vs Fake News Dataset, which consists of news articles categorized into two classes: genuine and fake.

3.2 Data Size:

- Total Samples: 20,000 news articles
- Number of genuine news articles: 10,000
- Number of fake news articles: 10,000
- Classes: Binary Classification (True and Fake)
- Features: Textual news articles

Dataset Split:

The dataset consists of 16,000 samples for training and 4,000 samples for testing.

The dataset was balanced with equal representation from both classes.

3.3. Preprocessing:

Various preprocessing methods were employed to clean and standardize the text data prior to model training.

- Lowercasing all text
- Expanding contractions
- Removing URLs
- Removing email addresses
- Removing mentions and hashtags
- Removing numbers
- Removing punctuation and special characters
- Removing extra whitespace
- Stop word removal
- Lemmatization

These preprocessing steps are standard practice for preparing text data for deep learning models (Goodfellow, Bengio and Courville, 2016).

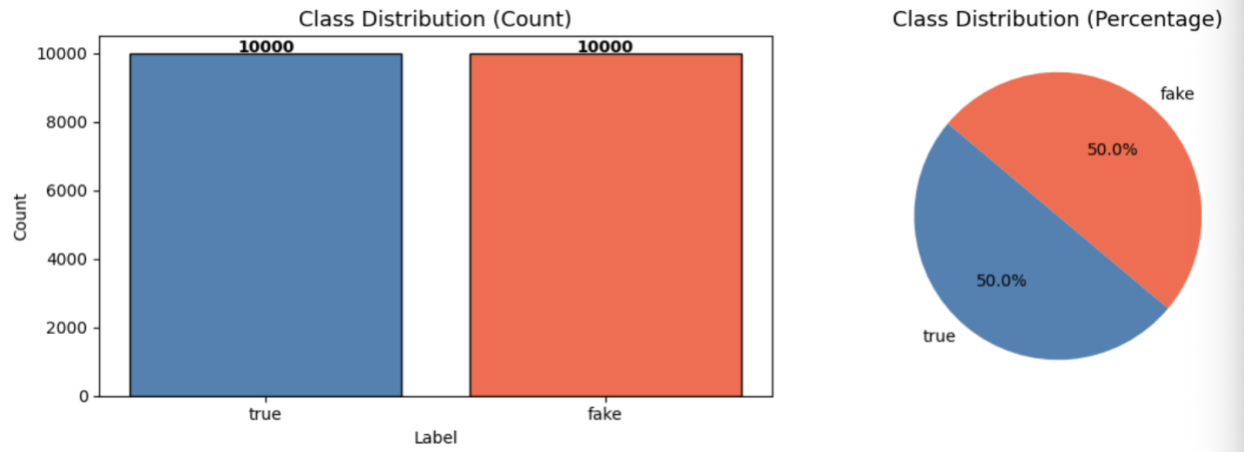


Figure 1 Class Distribution of True and Fake News Samples (Count and Percentage)



Figure 2 Word Cloud Representation of Most Frequent Words in True and Fake News Articles

To conduct a deeper investigation into the lexical properties of the data set, an approach based on frequency was applied to find out the most frequent words occurring in the true news and fake news articles. In this way, the observed differences from the word cloud can be validated numerically.

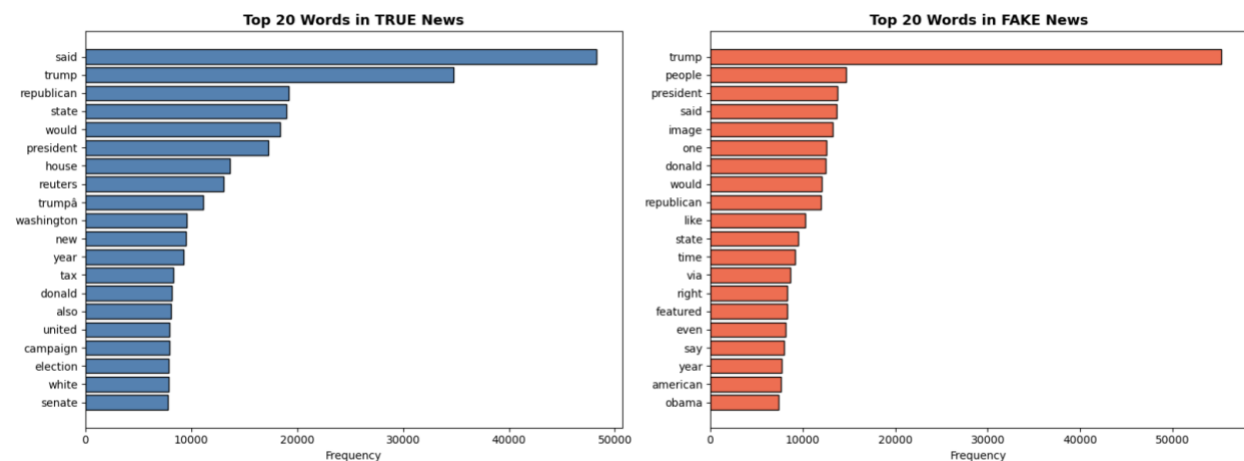


Figure 3 Top 20 Most Frequent Words in True and Fake News Articles After Preprocessing

After preprocessing the text data, Keras Tokenizer was used for tokenization. The vocabulary was restricted to the top 10,000 words, and all sequences were padded to a uniform length of 300 tokens to ensure consistent input size for all models.

The distribution of the number of words in the training data was investigated to set a suitable fixed value for the length of the inputs in our neural models. This process is important since deep learning models need inputs of a fixed size. It can also help select the optimal padding size to preserve most of the data.

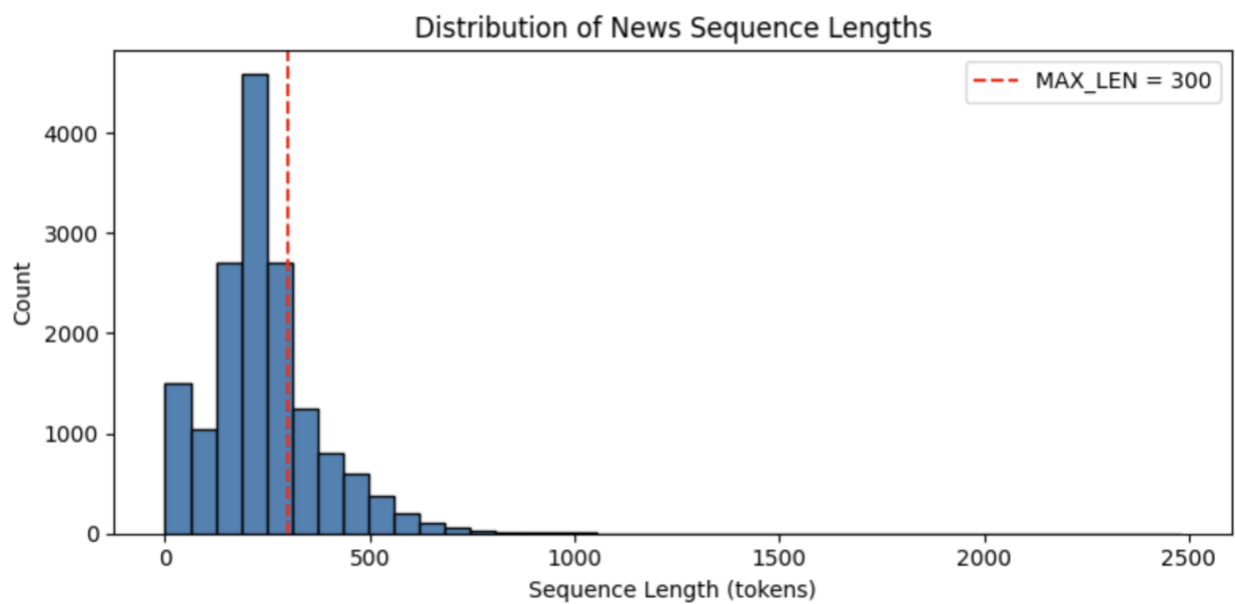


Figure 4 Distribution of News Article Sequence Lengths Before Padding

4. Methodology

4.1. Text Preprocessing:

The entire preprocessing pipeline was executed to convert the text data into numbers ready for processing through deep learning algorithms.

- a. The data was cleaned by eliminating irrelevant information and characters from the texts.
- b. The cleaned data was tokenized into integer sequences using Keras Tokenizer.
- c. Sequence padding was implemented to have an equal-length input vector across the data set.
- d. Class labels were encoded as:
 - Fake News: 0
 - True News: 1

4.2. Simple RNN model:

4.2.1. Model Architecture:

A 64-dimensional trainable embedding layer was used to represent words in dense vector form. This was followed by an LSTM layer with 64 units to capture sequential dependencies in the text. To reduce overfitting, a dropout layer with a rate of 0.3 was applied. The network then included a dense layer consisting of 32 neurons with ReLU activation for non-linear feature learning. Finally, a sigmoid-activated output layer was used to produce the final binary classification output.

4.2.2. Loss Function:

Binary Crossentropy Loss was used for binary classification tasks.

4.2.3. Optimizer:

The Adam optimizer was selected due to its efficient adaptive learning properties.

4.2.4. Hyperparameters:

- Embedding Dimension: 64
- Batch Size: 64
- Epochs: 20
- Dropout Rate: 0.3

4.3. LSTM Model

Developed to address the issue of vanishing gradient (Hochreiter and Schmidhuber, 1997)

4.3.1. Architecture

- a. Embedding Layer with 64-dimensional trainable embeddings.
- b. LSTM Layer consisting of 64 units.
- c. Dropout Layer with dropout rate of 0.3.
- d. Dense Layer with 32 neurons and ReLU activation.
- e. Output Layer with Sigmoid activation.

The LSTM architecture addresses the vanishing gradient problem through its gating mechanism, enabling effective learning over long sequences (Hochreiter and Schmidhuber, 1997).

4.3.2. Loss Function:

Binary Crossentropy Loss was used for binary classification.

4.3.3. Optimizer

Adam Optimizer was used with adaptive learning rate adjustments during training.

4.3.4. Hyperparameters

- Embedding Dimension: 64
- Batch Size: 64
- Epochs: 20
- Dropout Rate: 0.3

4.4. Transfer Learning with GloVe Embeddings

4.4.1. Pretrained Model

GloVe (Global Vectors for Word Representation)

4.4.2. Approach

- a. Pretrained GloVe embeddings were used to initialize the embedding layer. GloVe embeddings encode semantic relationships between words by training on global word co-occurrence statistics (Pennington, Socher and Manning, 2014).
- b. The embedding layer weights were frozen during training to reduce computational complexity.
- c. LSTM layers followed by dense layers were used for classification.

4.4.3. Loss Function

Binary Crossentropy Loss was used for binary classification tasks.

4.4.4. Optimizer

Adam Optimizer was used with adaptive learning rate updates.

4.4.5. Hyperparameters

- GloVe Embedding Dimension: 50
- Batch Size: 64
- Epochs: 20
- Dropout Rate: 0.3

5. Experiments and Results

5.1.RNN vs LSTM Performance

a. Simple RNN

- Accuracy: 94.42%
- Training Time: Lower compared to LSTM models
- Performance: Strong performance but struggled with long-term dependencies

b. LSTM

- Accuracy: 92.70%
- Training Time: Higher than Simple RNN due to increased complexity
- Performance: Better sequential learning capability but unstable validation behavior in this experiment.

c. LSTM with GloVe Embeddings

- Accuracy: 98.55%
- Performance: Best overall classification accuracy and contextual understanding
- Training Time: Efficient due to frozen pretrained embeddings

The LSTM + GloVe model achieved the highest accuracy, demonstrating the importance of pretrained embeddings for semantic understanding.



Figure 5 Training and Validation Accuracy and Loss for Simple RNN Model

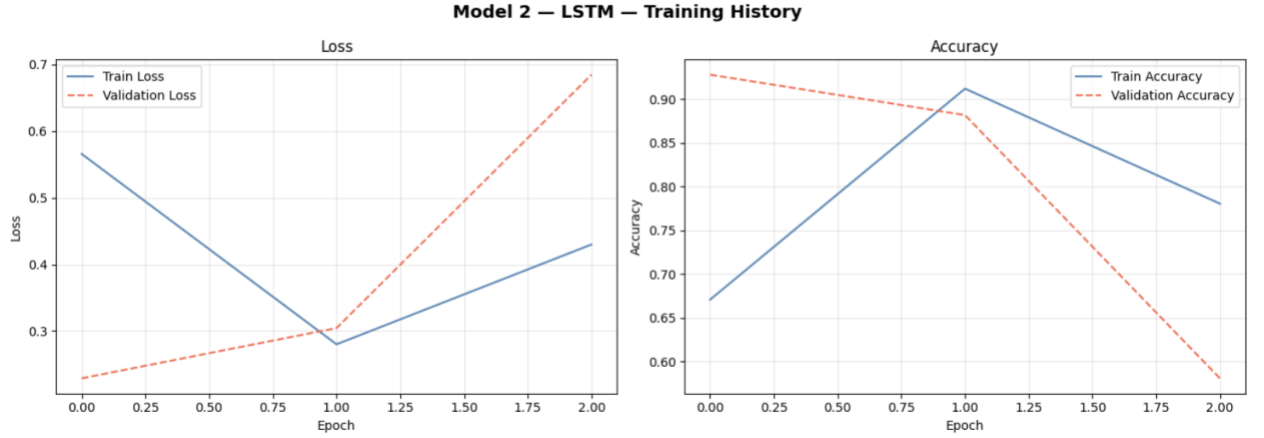


Figure 6 Training and Validation and Loss for LSTM Model



Figure 7 Training and Validation Accuracy and Loss for LSTM with GloVe Embeddings

5.2. Computational Efficiency

- **Simple RNN:** Low computational costs because of low training complexity and lower sequence computation. RNN-based architectures have been widely applied to sequential text classification tasks despite their limitations with long-term dependencies (Goodfellow, Bengio and Courville, 2016).
- **LSTM:** High computational costs due to the use of gates and higher parameter complexity.
- **LSTM with GloVe:** Fewer trainable parameters because the embedding layer is frozen, making training faster and improving convergence.

5.3. Training with Different Embeddings

a. Trainable Embeddings

- Applicable in Simple RNN and LSTM Models
- Generated semantic meanings based on the dataset
- Needed additional time for training to grasp context

b. Pretrained GloVe Embeddings

- Loaded with pretrained word vectors
- Covered vocabulary up to 96.9%
- Increased context knowledge and semantic connections
- Produced the best classification accuracy

The findings revealed that the use of pretrained embeddings greatly enhanced model performance in detecting fake news.

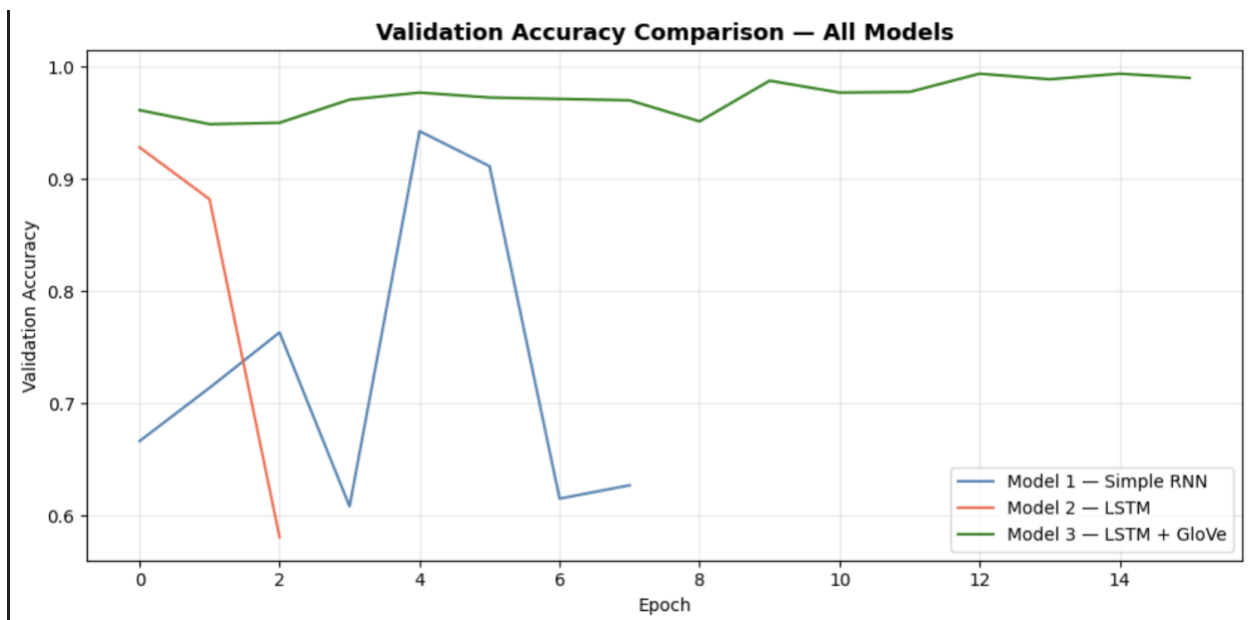


Figure 8 All 3 models validation accuracy on one plot

5.4. Model Evaluation

Metrics Used:

The effectiveness of the developed models was done using several standard classification metrics to provide a comprehensive performance analysis. The evaluation included:

- Accuracy- measures the proportion of correctly classified instances out of the total predictions made.
- Precision- indicates how many samples predicted as positive were positive.
- Recall- evaluates the model's ability to correctly identify all relevant positive instances.
- F1-Score- represents the harmonic mean of precision and recall, offering a balanced measure of classification quality.
- Confusion Matrix- provides a detailed breakdown of correct and incorrect predictions across different classes.

Simple RNN

- Accuracy: 94.42%
- Strong classification capability
- Minor instability during training

LSTM

- Accuracy: 92.70%
- Better long-term sequence understanding
- Validation loss increased quickly, causing early stopping

LSTM with GloVe

- Accuracy: 98.55%
- Best precision, recall, and F1-score
- Lowest classification error rate of 1.45%

The confusion matrix analysis showed that the LSTM with GloVe embeddings produced the most balanced predictions across both fake and true news categories.

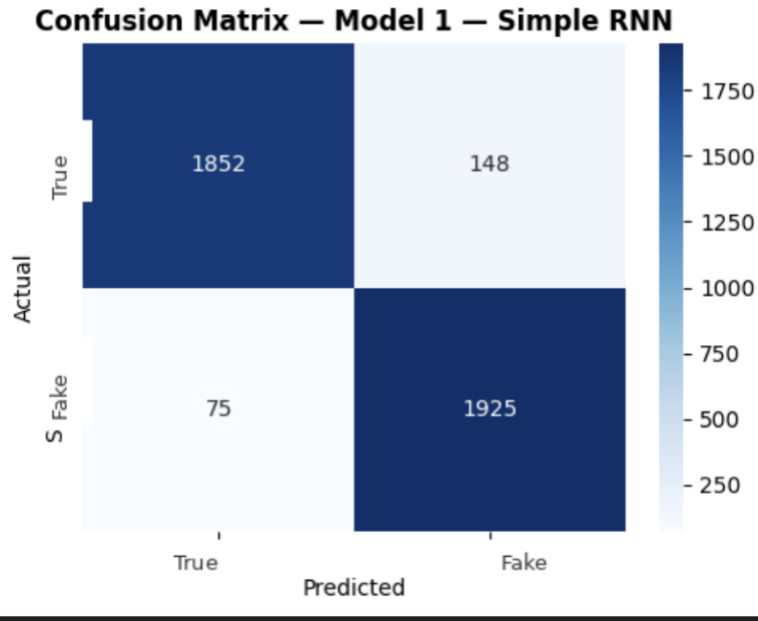


Figure 9 Prediction breakdown of the simple RNN showing correct and incorrect classification on the test

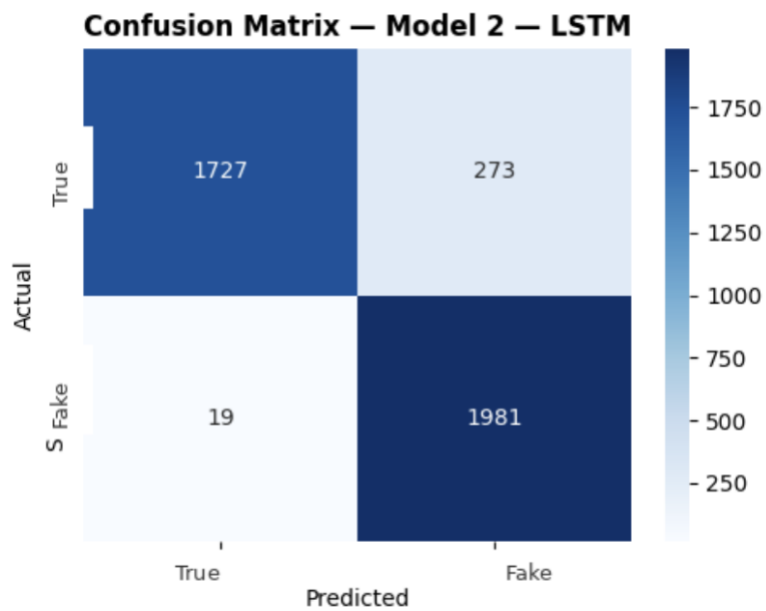


Figure 10 classification results of the LSTM illustrating its performance across true and fake news categories

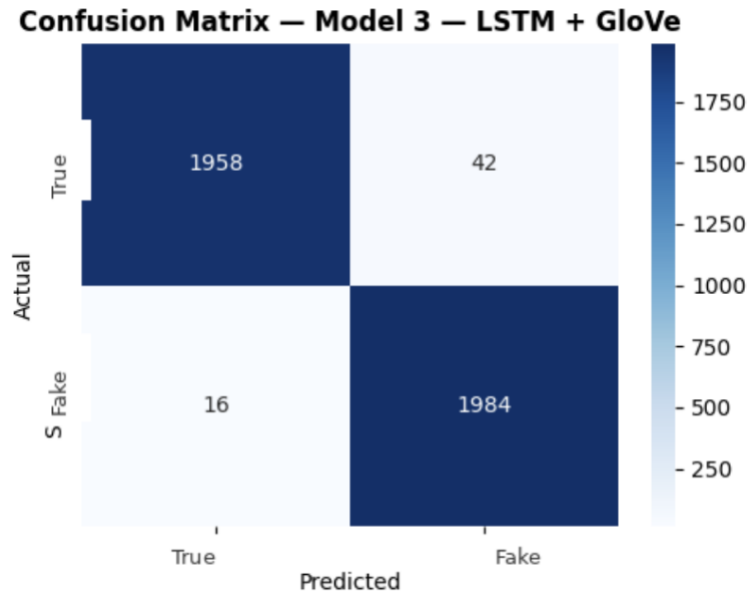


Figure 11 Confusion matrix for LSTM with GloVe Embeddings

Error analysis revealed that misclassified samples often contained:

- Ambiguous language
- Long and noisy articles
- Complex contextual meaning
- Unseen vocabulary

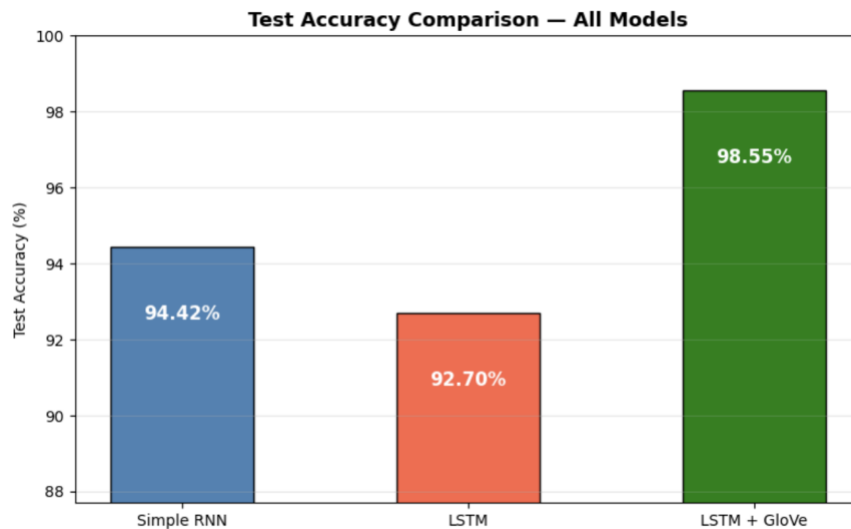


Figure 12 Test Accuracy chart

Such results highlight the advantage of transfer learning through pretrained embeddings in text classification tasks (Shu et al., 2017).

5.5. Graphical User Interface (GUI)

GUI using Gradio was implemented on the web to enable interaction with the three models. Any news article is pasted in the input field and gets instant results for predictions from all three models, showing TRUE/FAKE and their fake probability percentage results. There are four sample inputs provided already that can be tried out easily by users.

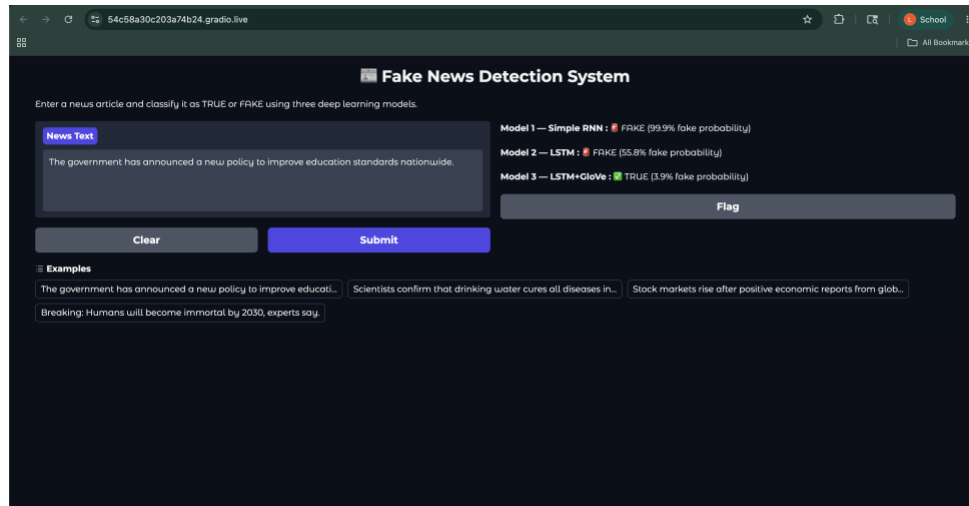


Figure 13 Gradio based Fake News Detection GUI showing article input and multi-model classification output

6. Conclusion and Future Work

6.1. Important Findings

- The Simple RNN model was computationally efficient but could not handle long dependencies.
- The LSTM network overcame the vanishing gradient problem to facilitate learning from sequence data.
- The LSTM network, using pretrained GloVe vectors, performed better than other models with an accuracy of 98.55%.
- Pretrained vectors helped to enhance the semantics and improve the performance of classification tasks.

6.2. Limitations

- The data may not encompass all possible variations in the language and writing style present in actual news articles.
- Hyperparameters and training settings affect model performance.
- Classification errors can happen even in articles that are extremely hard to classify based on context.

6.3. Future Work

- Employ Bidirectional LSTMs for context-sensitive learning.
- Do not freeze the embeddings but fine-tune them.
- Use attention mechanisms to enhance interpretability and model efficiency.
- Work with bigger and more varied datasets.

7. References

- Hochreiter, S. and Schmidhuber, J. (1997) ‘Long short-term memory’, *Neural Computation*, 9(8), pp. 1735–1780. Available at: <https://doi.org/10.1162/neco.1997.9.8.1735>
- Pennington, J., Socher, R. and Manning, C.D. (2014) ‘GloVe: Global vectors for word representation’, in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Doha: Association for Computational Linguistics, pp. 1532–1543. Available at: <https://doi.org/10.3115/v1/D14-1162>
- Shu, K. et al. (2017) ‘Fake news detection on social media: A data mining perspective’, *ACM SIGKDD Explorations Newsletter*, 19(1), pp. 22–36. Available at: <https://doi.org/10.1145/3137597.3137600>
- Goodfellow, I., Bengio, Y. and Courville, A. (2016) *Deep Learning*. Cambridge, MA: MIT Press. Available at: <https://www.deeplearningbook.org>
- Kingma, D.P. and Ba, J. (2015) ‘Adam: A method for stochastic optimization’, in *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*. San Diego. Available at: <https://arxiv.org/abs/1412.6980>

[NLP Task GitHub Link](#)

Text Classification Report

 Islington College, Nepal

Document Details

Submission ID

trn:oid::3618:138353335

15 Pages**Submission Date**

May 10, 2026, 1:31 PM GMT+5:45

1,686 Words**Download Date**

May 14, 2026, 12:16 PM GMT+5:45

13,689 Characters**File Name**

2408028_LunivaShrestha.pdf

File Size

419.1 KB

18% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Small Matches (less than 8 words)

Exclusions

- 1 Excluded Source

Match Groups

-  **11 Not Cited or Quoted 15%**
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations 0%**
Matches that are still very similar to source material
-  **2 Missing Citation 3%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 8%  Internet sources
- 4%  Publications
- 16%  Submitted works (Student Papers)

Handwritten signature